BTS SIO 2 – option SISR

Bloc n°3 – Sécurisation des serveurs et des systèmes

# TP n°4

# Haute disponibilité et répartition des charges d'un serveur web avec HAProxy

*Rapport de réalisation et guide technique*

Client : Les Jardins de Saint-Eloi

SRVHAPROXY / SRVWEB1 / SRVWEB2 – Debian 13 – HAProxy 3.0 – Apache 2

**Style Garraway**

BTS SIO 2 (option SISR) – Lycée Baimbridge

28 septembre 2026

# Sommaire

1. Contexte du projet ..... 3
2. Objectifs ..... 4
3. Prérequis et environnement ..... 5
3.1 Environnement prévu par l'énoncé et environnement utilisé ..... 5
3.2 Plan d'adressage ..... 6
3.3 Prérequis généraux ..... 6
4. Étapes de réalisation ..... 7
4.1 Étape 1 : reconfiguration réseau du cluster SRVWEB1 / SRVWEB2 ..... 7
4.2 Étape 2 : mise à jour d'Apache sur les deux serveurs web ..... 11
4.3 Étape 3 : installation et configuration de HAProxy sur SRVHAPROXY ..... 14
4.4 Étape 4 : tests de la répartition de charge ..... 24
5. Difficultés rencontrées et solutions ..... 29
6. Conclusion et perspectives ..... 30
7. Aide-mémoire des commandes ..... 31
8. Sitographie ..... 32

# 1. Contexte du projet

La société **Les Jardins de Saint-Eloi**, dirigée par Mme Bourgeois, vend des fleurs exotiques et des produits locaux de Guadeloupe. Son site web est hébergé sur des serveurs Linux Debian équipés d'Apache 2 et accessible à l'adresse `http://www.jardinsteloi.fr`.

Au TP n°3, Mme Bourgeois a obtenu un serveur de secours (cluster **actif/passif** avec Heartbeat) : un seul serveur travaille, l'autre attend la panne. Elle exprime maintenant un second besoin : que **chaque serveur web soit utilisé de manière optimale**, sans qu'un serveur soit surchargé pendant que l'autre est sous-utilisé.

Ma mission de stagiaire est de mettre en place un **répartiteur de charge** (load balancer) avec **HAProxy** devant les deux serveurs web SRVWEB1 et SRVWEB2, qui fonctionneront désormais en mode **actif/actif**, puis de démontrer par des tests que la charge est répartie et que le site reste disponible si un serveur tombe.

## 2. Objectifs

- isoler SRVWEB1 (`10.0.0.2`) et SRVWEB2 (`10.0.0.3`) dans un réseau privé « HA » (`10.0.0.0/24`) ;
- créer un serveur SRVHAPROXY relié à la fois au réseau des clients (`192.168.60.4`) et au réseau HA (`10.0.0.1`) ;
- installer et configurer HAProxy pour répartir les requêtes HTTP entre les deux serveurs (algorithme *round robin*) ;
- superviser le service grâce à la page de statistiques de HAProxy ;
- prouver par deux scénarios de test que la charge est répartie, puis que le site reste accessible quand un serveur web est arrêté ;
- documenter l'ensemble pour qu'un technicien débutant puisse le refaire seul.

**Vocabulaire indispensable :**

| Terme | Définition simple | Exemple dans ce TP |
|---|---|---|
| Répartition de charge (load balancing) | Distribuer les requêtes entre plusieurs serveurs pour qu'aucun ne soit surchargé. | 1re visite vers SRVWEB1, 2e vers SRVWEB2, etc. |
| Proxy inverse (reverse proxy) | Serveur placé devant les serveurs web : il reçoit les requêtes des clients et les transmet en coulisse. Le client ne voit jamais les vrais serveurs. | SRVHAPROXY reçoit les requêtes sur `192.168.60.4`. |
| Actif/actif | Tous les serveurs travaillent en même temps (différent de l'actif/passif du TP3). | SRVWEB1 et SRVWEB2 répondent tous les deux. |
| Frontend | Partie de HAProxy tournée vers les clients : « par où j'écoute ». | `frontend site_web`, port 80 |
| Backend | Partie de HAProxy tournée vers les serveurs web : « à qui je transmets ». | `backend serveurs` |
| Round robin | Algorithme « chacun son tour », dans l'ordre, en boucle. | n°1, n°2, n°1, n°2… |
| Health check | Vérification régulière qu'un serveur répond ; s'il tombe, HAProxy cesse de lui envoyer des requêtes. | Paramètre `check` |
| LAN Segment (VMware) | Réseau virtuel privé : seules les VM qui y sont branchées peuvent communiquer. | Segment « HA » |

# 3. Prérequis et environnement

## 3.1 Environnement prévu par l'énoncé et environnement utilisé

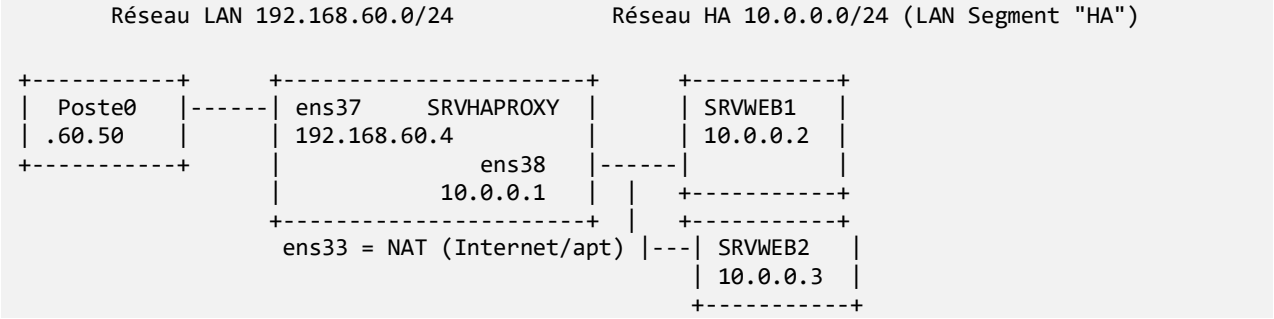
L'environnement réel diffère de l'énoncé sur plusieurs points, recensés et justifiés ci-dessous. **Ce sont les valeurs réelles qui sont utilisées dans tout le document.**

| Élément | Prévu par l'énoncé | Utilisé réellement | Justification |
|---|---|---|---|
| Hyperviseur | VMware Workstation professionnel | VMware Workstation Pro | Conforme. |
| Système des serveurs | Debian Trixie (annexe : Debian 12.0) | Debian GNU/Linux 13 (trixie), noyau 6.12.107+deb13-amd64 | Relevé par `hostnamectl` (figure 10). L'annexe mentionne Debian 12 par erreur. |
| Version de HAProxy | Non précisée | 3.0.11-1+deb13u3 | Version fournie par les dépôts Debian 13 (figures 17 et 20). |
| Poste client | poste0 / poste1 | Poste0 (Windows), adresse 192.168.60.50 | Un seul poste suffit aux tests. Adresse relevée au TP3. |
| Valeur « X » du réseau | 192.168.X.0/24 | 192.168.60.0/24 | Numéro de réseau attribué. |
| Passerelle LAN | 192.168.X.1 sur SRVHAPROXY | Aucune passerelle sur `ens37` | Une machine ne doit avoir qu'une passerelle par défaut : celle de `ens33` (NAT, DHCP), nécessaire à `apt`. |
| Switch « HA » | Switch réseau interne HA | LAN Segment VMware nommé « HA » | Équivalent virtuel d'un switch isolé (figure 5). |
| Cartes réseau | Non nommées | SRVWEB : `ens33` (NAT) + `ens37` (HA)<br>SRVHAPROXY : `ens33` (NAT) + `ens37` (LAN) + `ens38` (HA) | Nommage VMware (VirtualBox utiliserait `enp0s3`). |
| Fichier de site Apache | `sainteloi.conf` | `jardinsteloi.conf` (créé au TP3) | Le fichier du TP3 convient sans modification : `<VirtualHost *:80>` accepte toute adresse. |
| Cluster Heartbeat du TP3 | — | Arrêté et désactivé | Sinon SRVWEB1 reprendrait la 192.168.60.4, désormais portée par SRVHAPROXY (conflit d'adresse). |
| Installation de HAProxy | « sur chaque serveur » | Uniquement sur SRVHAPROXY | Les serveurs web n'ont pas besoin de HAProxy ; il n'est utile que sur le répartiteur. |
| Port | 80 (ou 443) | 80 | Le site est en HTTP, aucun certificat HTTPS n'est installé. |
| Test des serveurs web | Depuis un poste de travail | Depuis SRVHAPROXY avec `curl` | Le réseau HA est isolé : Poste0 ne peut pas joindre 10.0.0.2 ni 10.0.0.3, ce qui est voulu. |

## 3.2 Plan d'adressage

| Machine | Rôle | Carte | Adresse IP / masque |
|---|---|---|---|
| SRVWEB1 | Serveur web n°1 | ens37 (HA)<br>ens33 (NAT) | 10.0.0.2/24<br>DHCP (192.168.223.x) |
| SRVWEB2 | Serveur web n°2 | ens37 (HA)<br>ens33 (NAT) | 10.0.0.3/24<br>DHCP (192.168.223.x) |
| SRVHAPROXY | Répartiteur de charge | ens37 (LAN)<br>ens38 (HA)<br>ens33 (NAT) | 192.168.60.4/24<br>10.0.0.1/24<br>DHCP (192.168.223.152) |
| Poste0 | Client de test | LAN | 192.168.60.50/24 |
| Nom du site | www.jardinsteloi.fr | — | résolu en 192.168.60.4 |

*Schéma simplifié de l'infrastructure (mode actif/actif)*




## 3.3 Prérequis généraux

- Les serveurs SRVWEB1 et SRVWEB2 du TP n°3 sont fonctionnels (Apache 2, site dans `/var/www/sainteloi`).
- La VM de sauvegarde `SRVWEB_CLONE` du TP n°3 existe (elle servira à créer SRVHAPROXY) et doit rester **éteinte** pendant le TP.
- Le mot de passe **root** est connu ; les commandes système s'exécutent en root après `su -`.
- Le poste hôte peut faire tourner 4 VM en même temps (SRVWEB1, SRVWEB2, SRVHAPROXY, Poste0).
- Un snapshot « Fin TP3 » de SRVWEB1 et SRVWEB2 a été pris dans VMware avant de commencer (retour arrière possible).

**Convention d'écriture.** Ne tapez jamais le caractère `#` ou `$` qui termine l'invite de commande (par exemple `root@SRVHAPROXY:~#`) : il indique seulement le compte utilisé (`#` = root, `$` = utilisateur normal). Chaque bloc de code indique en titre la machine sur laquelle l'exécuter.

# 4. Étapes de réalisation

## 4.1 Étape 1 : reconfiguration réseau du cluster SRVWEB1 / SRVWEB2

### a) Objectif

Placer les deux serveurs web dans le réseau privé HA (`10.0.0.0/24`) et vérifier qu'ils communiquent entre eux par leur nom. Le clonage de SRVWEB en SRVWEB1 et SRVWEB2, demandé par l'énoncé, a déjà été fait au TP n°3.

### b) Prérequis

- TP n°3 terminé, snapshot « Fin TP3 » pris.
- VM SRVWEB1 et SRVWEB2 démarrées, session root (`su -`).

### c) Procédure

1. **Sur SRVWEB1 et SRVWEB2**, arrêtez le cluster Heartbeat du TP3 et redonnez à Apache son démarrage automatique :

*SRVWEB1 et SRVWEB2*

```
systemctl stop heartbeat
systemctl disable heartbeat
systemctl enable apache2
systemctl start apache2
```

2. **Dans VMware**, pour SRVWEB1 puis SRVWEB2 : **VM > Settings > Network Adapter 2** (celui de `ens37`). Choisissez **LAN Segment**, cliquez sur **LAN Segments… > Add**, nommez-le `HA`, sélectionnez-le, et cochez **Connected** et **Connect at power on**. Laissez la carte 1 en **NAT**.
3. **Sur SRVWEB1**, modifiez le bloc `ens37` du fichier de configuration réseau (contenu en d) :

*SRVWEB1*

```
nano /etc/network/interfaces
```

4. **Sur SRVWEB2**, faites de même avec l'adresse `10.0.0.3` :

*SRVWEB2*

```
nano /etc/network/interfaces
```

5. Appliquez la configuration et vérifiez :

*SRVWEB1 et SRVWEB2*

```
systemctl restart networking
ip a show ens37
```

6. Éditez l'annuaire local des noms (contenu en d) :

*SRVWEB1 et SRVWEB2*

```
nano /etc/hosts
```

7. Testez la communication par nom :

*SRVWEB1*

```
ping -c 4 SRVWEB2
```

*SRVWEB2*

```
ping -c 4 SRVWEB1
```

### d) Contenu des fichiers

*SRVWEB1 : seul le bloc ens37 change (auparavant 192.168.60.14). Sur SRVWEB2 : address 10.0.0.3 (auparavant 192.168.60.15)*

```
# Bloc ens37 de /etc/network/interfaces - SRVWEB1
auto ens37
iface ens37 inet static
    address 10.0.0.2
    netmask 255.255.255.0
```

*/etc/hosts sur SRVWEB1*

```
127.0.0.1   localhost
127.0.1.1   SRVWEB1
10.0.0.3    SRVWEB2
```

*/etc/hosts sur SRVWEB2*

```
127.0.0.1   localhost
127.0.1.1   SRVWEB2
10.0.0.2    SRVWEB1
```

**Ce qui a changé par rapport au TP3 :** l'adresse de `ens37` passe du réseau clients (192.168.60.x) au réseau HA (10.0.0.x). Le bloc `ens33` (DHCP, NAT) ne change pas. Il n'y a toujours pas de ligne `gateway` sur `ens37`.

### e) Réponses aux questions de l'énoncé

**Procédure de modification du plan d'adressage :** décrite en c). **Rôle du fichier `/etc/hosts` :** c'est un annuaire local qui associe des noms de machines à des adresses IP, consulté **avant** le DNS. Il permet de joindre l'autre serveur par son nom même si le DNS est indisponible.

| Ligne | Signification |
|---|---|
| `127.0.0.1 localhost` | Adresse de boucle locale : la machine se désigne elle-même. |
| `127.0.1.1 SRVWEB1` | Convention Debian : associe le nom de la machine à elle-même. |
| `10.0.0.3 SRVWEB2` | Adresse de l'autre serveur, dans le réseau HA, pour le joindre par son nom (`10.0.0.2 SRVWEB1` sur SRVWEB2). |

### f) Résultat attendu

`ip a show ens37` affiche **state UP** et `10.0.0.2/24` (SRVWEB1) ou `10.0.0.3/24` (SRVWEB2). Chaque `ping` reçoit 4 réponses, sans perte.

### g) Captures

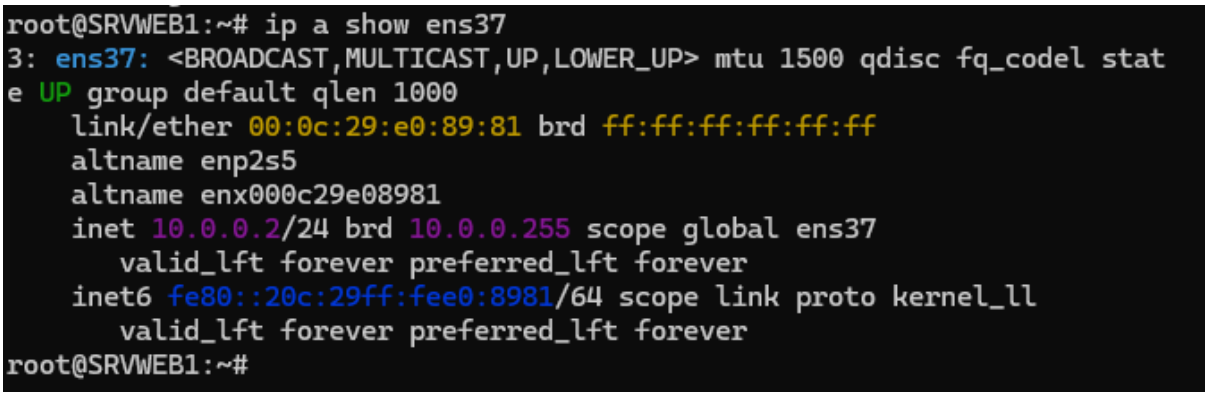



*Figure 1 – Adresse IP de SRVWEB1 sur le réseau HA*

**Ce que montre la capture.** Sur SRVWEB1 (invite `root@SRVWEB1`), `ens37` est **UP**, MAC `00:0c:29:e0:89:81`, adresse `10.0.0.2/24`, diffusion `10.0.0.255`.

**Rôle de l'étape.** Placer SRVWEB1 dans le réseau HA avec l'adresse imposée par le document 1 de l'énoncé.

**Ce que cela valide.** SRVWEB1 est correctement adressé sur le réseau HA.

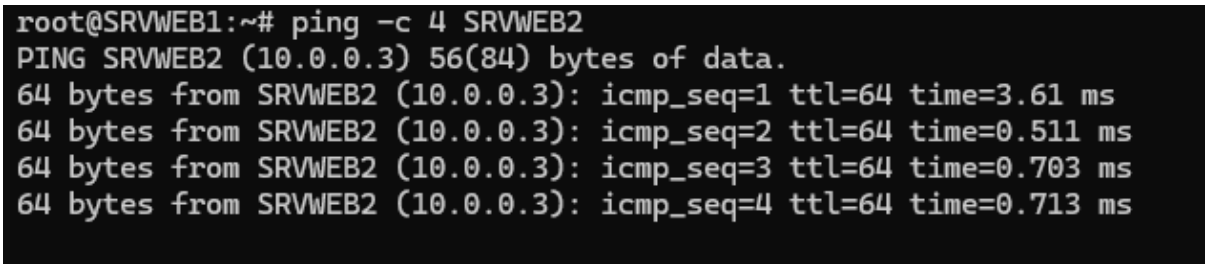



*Figure 2 – Ping de SRVWEB2 par son nom depuis SRVWEB1*

**Ce que montre la capture.** `ping -c 4 SRVWEB2` : le nom est résolu en `10.0.0.3` et 4 réponses arrivent (`ttl=64`, de 0,511 à 3,61 ms). Les lignes de statistiques finales ne sont pas visibles.

**Rôle de l'étape.** Vérifier la communication réseau et la résolution de nom par `/etc/hosts`.

**Ce que cela valide.** SRVWEB1 joint SRVWEB2 à l'adresse 10.0.0.3 : le réseau HA fonctionne dans ce sens.

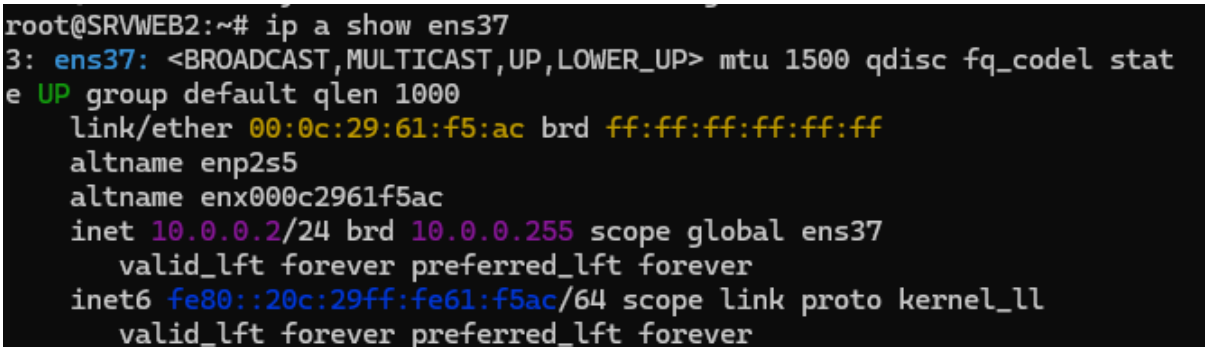



*Figure 3 – Adresse IP de SRVWEB2 sur le réseau HA*

**Ce que montre la capture.** Sur SRVWEB2 (`root@SRVWEB2`), `ens37` est **UP**, MAC `00:0c:29:61:f5:ac`, mais l'adresse affichée est **`10.0.0.2/24`**.

**Rôle de l'étape.** Attribuer à SRVWEB2 l'adresse `10.0.0.3`.

**Ce que cela valide. Anomalie signalée.** L'adresse attendue est `10.0.0.3` ; `10.0.0.2` est celle de SRVWEB1, ce qui crée un conflit d'adresse. Or les figures 2, 9 et 20 montrent que SRVWEB2 répond bien à `10.0.0.3` : la capture a probablement été prise avant correction, ou l'adresse a été corrigée sans nouvelle capture. Le conflit possible rend aussi peu concluante la figure suivante.

```
root@SRVWEB2:~# ping -c 4 SRVWEB1
PING SRVWEB1 (10.0.0.2) 56(84) bytes of data.
64 bytes from SRVWEB1 (10.0.0.2): icmp_seq=1 ttl=64 time=2.22 ms
64 bytes from SRVWEB1 (10.0.0.2): icmp_seq=2 ttl=64 time=0.663 ms
64 bytes from SRVWEB1 (10.0.0.2): icmp_seq=3 ttl=64 time=0.726 ms
64 bytes from SRVWEB1 (10.0.0.2): icmp_seq=4 ttl=64 time=1.43 ms
```

*Figure 4 – Ping de SRVWEB1 par son nom depuis SRVWEB2*

**Ce que montre la capture.** `ping -c 4 SRVWEB1` : le nom est résolu en `10.0.0.2` et 4 réponses arrivent (`ttl=64`, de 0,663 à 2,22 ms).

**Rôle de l'étape.** Vérifier la communication dans l'autre sens.

**Ce que cela valide.** Les temps (de l'ordre de 0,7 ms) sont ceux d'une machine distante, mais l'anomalie de la figure 3 impose de refaire ce test après contrôle de l'adresse de SRVWEB2.

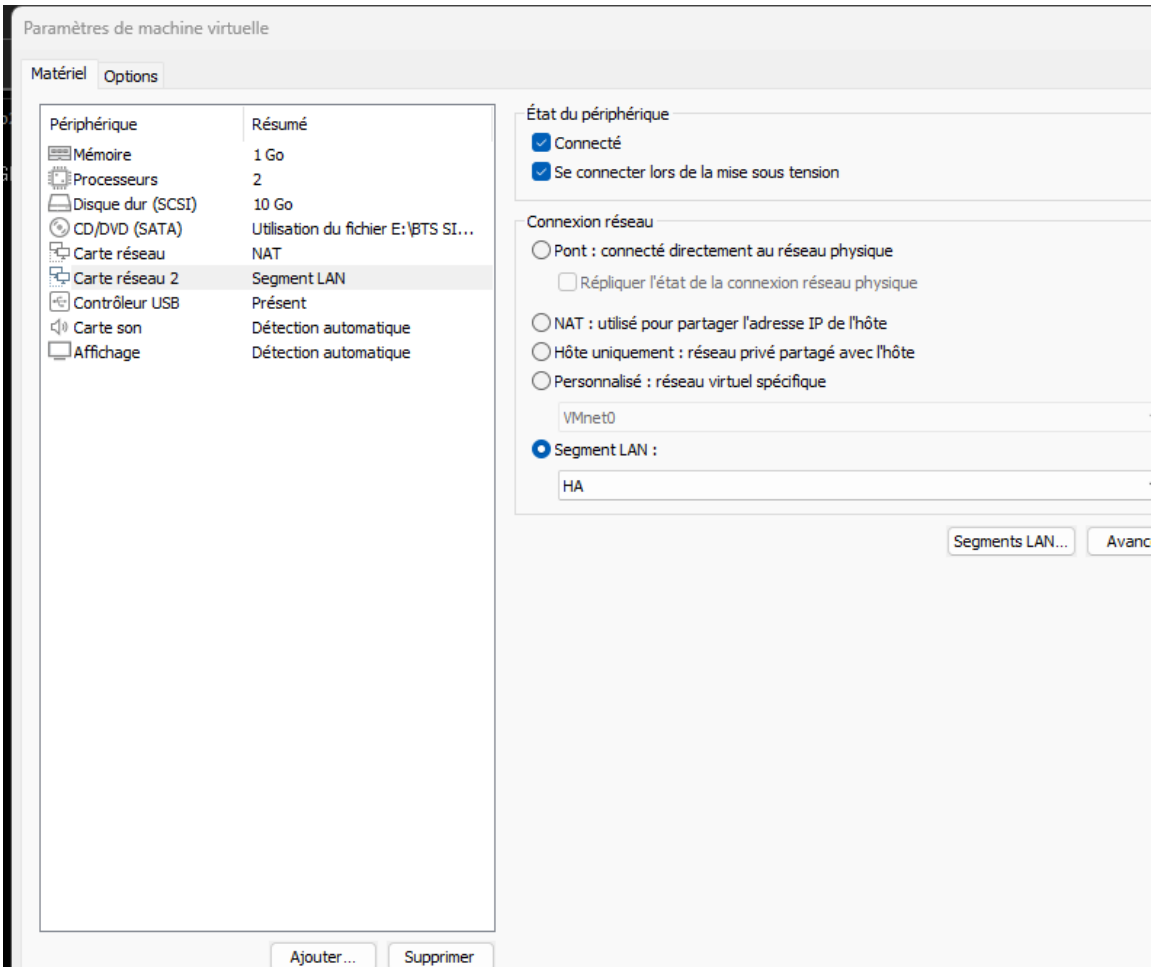



*Figure 5 – Paramètres VMware : carte réseau 2 sur le segment LAN « HA »*

**Ce que montre la capture.** La fenêtre « Paramètres de machine virtuelle » indique : mémoire 1 Go, 2 processeurs, disque 10 Go, **Carte réseau** en NAT, **Carte réseau 2** en **Segment LAN**. Pour la carte 2, les cases **Connecté** et **Se connecter lors de la mise sous tension** sont cochées, et le segment sélectionné est **HA**. Le nom de la VM n'est pas visible.

**Rôle de l'étape.** Relier la carte du réseau privé au segment HA, tout en gardant le NAT pour l'accès Internet.

**Ce que cela valide.** Le réseau HA isolé est en place et la carte est connectée. La même vérification vaut pour les autres VM du réseau HA.

### h) Point de contrôle

*SRVWEB1 (sur SRVWEB2 : ping -c 4 SRVWEB1)*

```
uname -n
ip -br a show ens37
ping -c 4 SRVWEB2
```

Attendu : le bon nom, `ens37 UP` avec la bonne adresse (10.0.0.2 sur SRVWEB1, **10.0.0.3** sur SRVWEB2), 4 réponses.

### i) En cas de problème

| Symptôme | Cause probable | Diagnostic / correction |
|---|---|---|
| `ens37` en DOWN / NO-CARRIER | Carte déconnectée ou mauvais segment dans VMware | Cocher **Connected** ; vérifier que le segment est bien HA (même nom partout). |
| Ping : `Destination Host Unreachable` | Mauvaise adresse, conflit d'IP ou VM sur un autre segment | `ip -br a` sur chaque machine ; adresses uniques. |
| Heartbeat reprend la .4 ou l'IP virtuelle | Heartbeat encore actif | `systemctl disable --now heartbeat`. |
| Le nom n'est pas résolu | `/etc/hosts` incomplet | `cat /etc/hosts`. |

## 4.2 Étape 2 : mise à jour d'Apache sur les deux serveurs web

### a) Objectif

S'assurer que chaque serveur sert son propre site et affiche une page différente (n°1 ou n°2). Cette différence volontaire permet de **voir** quel serveur a répondu lors des tests de répartition.

### b) Prérequis

- Étape 1 validée.
- Apache actif sur les deux serveurs (`systemctl is-active apache2`).
- SRVHAPROXY créé (étape 3) pour le test avec `curl`, car Poste0 n'a pas accès au réseau HA.

### c) Procédure

1. Vérifiez le fichier de site actif (déjà configuré au TP3, aucune modification nécessaire ici) :

*SRVWEB1 et SRVWEB2*

```
apache2ctl configtest
grep -E "VirtualHost|ServerName" /etc/apache2/sites-available/jardinsteloi.conf
```

2. Corrigez la page d'accueil de **SRVWEB2**, qui affichait encore l'ancienne page au TP3 :

*SRVWEB2*

```
nano /var/www/sainteloi/index.html
```

3. Vérifiez le contenu des deux pages :

*SRVWEB1 puis SRVWEB2*

```
cat /var/www/sainteloi/index.html
```

4. Installez `curl` (navigateur en ligne de commande, il affiche le code HTML renvoyé par un serveur) puis testez chaque serveur :

*SRVHAPROXY*

```
apt install curl -y
curl http://10.0.0.2
curl http://10.0.0.3
```

### d) Contenu des fichiers

*/var/www/sainteloi/index.html sur SRVWEB2 (sur SRVWEB1 : « site n°1 »)*

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8">
    <title>Site - jardins de Saint-Eloi</title>
  </head>
  <body>
    <h1>Bienvenue sur le site n°2 des Jardins de Saint-Eloi</h1>
    <p>Vente de fleurs exotiques et de produits locaux de Guadeloupe.</p>
  </body>
</html>
```

**Ce qui a changé :** la balise `<h1>` de SRVWEB2 affiche désormais « site n°2 ». Le fichier `jardinsteloi.conf` du TP3 est conservé tel quel : `<VirtualHost *:80>` accepte les requêtes sur toute adresse, donc aussi sur `10.0.0.2` et `10.0.0.3`. L'énoncé cite `sainteloi.conf`, ancien nom du même fichier.

### e) Réponses aux questions de l'énoncé

**Paramètres modifiés ou ajoutés dans le fichier de configuration du site :** aucun. Les paramètres du TP3 (`ServerName jardinsteloi.fr`, `ServerAlias www.jardinsteloi.fr`, `DocumentRoot /var/www/sainteloi`, `Require all granted`, journaux `error.log` et `access.log`) sont suffisants. **Test depuis un poste de travail :** Poste0 ne peut pas joindre le réseau HA (isolation voulue, argument de sécurité) ; le test s'effectue donc depuis SRVHAPROXY, seule machine reliée aux deux réseaux, avec `curl`.

### f) Résultat attendu

`curl http://10.0.0.2` renvoie le code HTML contenant « site n°1 » ; `curl http://10.0.0.3` renvoie « site n°2 ».

### g) Captures

```
user@SRVWEB2:~$ cat /var/www/sainteloi/index.html
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8">
    <title>Site - jardins de Saint-Eloi</title>
  </head>
  <body>
    <h1>Bienvenue sur le site n°2 des Jardins de Saint-Eloi</h1>
    <p>Vente de fleurs exotiques et de produits locaux de Guadeloupe.</p>
  </body>
</html>
user@SRVWEB2:~$
```

*Figure 6 – Page d'accueil de SRVWEB2*

**Ce que montre la capture.** `cat /var/www/sainteloi/index.html` sur SRVWEB2 (invite `user@SRVWEB2:~$`) : `<title>Site - jardins de Saint-Eloi</title>`, `<h1>Bienvenue sur le site n°2 des Jardins de Saint-Eloi</h1>` et un paragraphe sur la vente de fleurs exotiques.

**Rôle de l'étape.** Corriger la page de SRVWEB2 afin qu'elle se distingue de celle de SRVWEB1.

**Ce que cela valide.** La page n°2 est en place. Elle corrige l'écart constaté au TP3, où SRVWEB2 servait encore l'ancienne page. La lecture d'un fichier ne demande pas les droits root.

```
user@SRVWEB1:~$ cat /var/www/sainteloi/index.html
<html><head><meta charset="utf-8"><title>Site - jardins de Saint-Eloi</title></head><bo
dy><h1>Bienvenue sur le site n°1 des Jardins de Saint-Eloi</h1></body></html>
user@SRVWEB1:~$
```

*Figure 7 – Page d'accueil de SRVWEB1*

**Ce que montre la capture.** `cat /var/www/sainteloi/index.html` sur SRVWEB1 : le même contenu HTML, écrit sur une seule ligne, avec `<h1>Bienvenue sur le site n°1 des Jardins de Saint-Eloi</h1>`.

**Rôle de l'étape.** Vérifier la page du serveur principal.

**Ce que cela valide.** La page n°1 est en place. La mise en forme diffère de celle de SRVWEB2 (ligne unique, pas de paragraphe) sans conséquence sur les tests ; l'harmoniser rendrait les deux serveurs plus homogènes.

```
root@SRVHAPROXY:~# curl http://10.0.0.2
<html><head><meta charset="utf-8"><title>Site - jardins de Saint-Eloi</title></head><bo
dy><h1>Bienvenue sur le site n°1 des Jardins de Saint-Eloi</h1></body></html>
root@SRVHAPROXY:~#
```

*Figure 8 – Test de SRVWEB1 depuis SRVHAPROXY (curl)*

**Ce que montre la capture.** Sur SRVHAPROXY, `curl http://10.0.0.2` renvoie le code HTML avec « Bienvenue sur le site n°1 des Jardins de Saint-Eloi ».

**Rôle de l'étape.** Vérifier qu'Apache de SRVWEB1 répond à travers le réseau HA.

**Ce que cela valide.** SRVHAPROXY atteint SRVWEB1 en HTTP : le futur backend `srvweb1` sera joignable.

```
root@SRVHAPROXY:~# curl http://10.0.0.3
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8">
    <title>Site - jardins de Saint-Eloi</title>
  </head>
  <body>
    <h1>Bienvenue sur le site n°2 des Jardins de Saint-Eloi</h1>
    <p>Vente de fleurs exotiques et de produits locaux de Guadeloupe.</p>
  </body>
</html>
root@SRVHAPROXY:~#
```

*Figure 9 – Test de SRVWEB2 depuis SRVHAPROXY (curl)*

**Ce que montre la capture.** `curl http://10.0.0.3` renvoie la page complète avec « Bienvenue sur le site n°2 des Jardins de Saint-Eloi » et le paragraphe sur la vente de fleurs.

**Rôle de l'étape.** Vérifier qu'Apache de SRVWEB2 répond à travers le réseau HA.

**Ce que cela valide.** SRVWEB2 répond bien à l'adresse `10.0.0.3` : elle lève le doute posé par la figure 3.

### h) Point de contrôle

*SRVHAPROXY*

```
curl -s http://10.0.0.2 | grep h1
curl -s http://10.0.0.3 | grep h1
```

Attendu : « site n°1 » puis « site n°2 ».

### i) En cas de problème

| Symptôme | Cause probable | Diagnostic / correction |
|---|---|---|
| `curl` : Connection refused | Apache arrêté sur le serveur visé | `systemctl start apache2` ; `systemctl enable apache2`. |
| `curl` : No route to host / timeout | Réseau HA mal branché ou mauvaise adresse | `ping -c 4 10.0.0.2` ; `ip -br a`. |
| Mauvaise page affichée | `index.html` non modifié | `cat /var/www/sainteloi/index.html`. |
| Page par défaut d'Apache | Site `000-default` actif | `a2dissite 000-default.conf` puis `systemctl reload apache2`. |

## 4.3 Étape 3 : installation et configuration de HAProxy sur SRVHAPROXY

### a) Objectif

Créer le répartiteur de charge SRVHAPROXY, le relier aux deux réseaux, installer HAProxy et le configurer pour répartir les requêtes entre SRVWEB1 et SRVWEB2. C'est le cœur du TP.

### b) Prérequis

- Étapes 1 et 2 validées.
- VM `SRVWEB_CLONE` (sauvegarde du TP3, sans Heartbeat) éteinte, prête à être clonée.
- Accès Internet via `ens33` pour `apt`.

### c) Procédure

**A. Création de la VM.**

1. **Dans VMware**, avec `SRVWEB_CLONE` éteinte : **VM > Manage > Clone**, « The current state », **Create a full clone** (clone intégral), nom `SRVHAPROXY`.
2. **Dans VMware**, sur `SRVHAPROXY` : **Settings > Network Adapter > Advanced > Generate** pour chaque carte (nouvelles adresses MAC).
3. **Dans VMware**, configurez trois cartes : **Adapter 1** en NAT ; **Adapter 2** sur le réseau de Poste0 (LAN) ; **Adapter 3** (bouton **Add… > Network Adapter**) sur le **LAN Segment HA**. Cochez **Connected** et **Connect at power on** pour chacune.
4. Démarrez la VM, passez en root et renommez la machine :

*SRVHAPROXY*

```
su -
hostnamectl set-hostname SRVHAPROXY
hostnamectl
ip a
```

5. Éditez la configuration réseau (contenu complet en d) :

*SRVHAPROXY*

```
nano /etc/network/interfaces
```

6. Appliquez et vérifiez (le redémarrage efface les anciennes adresses) :

*SRVHAPROXY*

```
systemctl restart networking
systemctl status networking --no-pager
ip -br a
ip route
```

7. Éditez `/etc/hosts` (contenu en d) puis testez les serveurs web :

*SRVHAPROXY*

```
nano /etc/hosts
```

*SRVHAPROXY*

```
ping -c 4 SRVWEB1
ping -c 4 SRVWEB2
```

**B. Installation de HAProxy.**

1. Le clone contient Apache, qui occuperait le port 80 dont HAProxy a besoin : arrêtez-le et désactivez-le.

*SRVHAPROXY*

```
systemctl stop apache2
systemctl disable apache2
```

2. Mettez à jour le système et installez HAProxy :

*SRVHAPROXY*

```
apt update
apt upgrade -y
apt install haproxy -y
```

3. Activez le service au démarrage et affichez son état :

*SRVHAPROXY*

```
systemctl enable haproxy
systemctl status haproxy
```

4. Faites une copie de sauvegarde du fichier de configuration et vérifiez-la :

*SRVHAPROXY*

```
cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.save
ls /etc/haproxy/
```

5. Ajoutez les trois sections de HAProxy **à la fin** de `haproxy.cfg`, sans toucher au reste (contenu en d) :

*SRVHAPROXY*

```
nano /etc/haproxy/haproxy.cfg
```

6. Vérifiez la validité de la configuration (tirets **normaux** `-c -f`, jamais des tirets longs copiés depuis le PDF, et **avec le chemin du fichier**) :

*SRVHAPROXY*

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

7. Relancez le service pour prendre en compte la configuration, puis vérifiez :

*SRVHAPROXY*

```
systemctl restart haproxy
systemctl status haproxy
```

## d) Contenu des fichiers

*SRVHAPROXY : pas de ligne gateway (Internet passe par ens33) ; aucune adresse dans les deux cartes du même sous-réseau*

```
# /etc/network/interfaces - SRVHAPROXY (fichier complet)
source /etc/network/interfaces.d/*

auto lo
iface lo inet loopback

# ens33 : NAT, acces Internet (apt)
allow-hotplug ens33
iface ens33 inet dhcp

# ens37 : reseau LAN, cote clients
auto ens37
iface ens37 inet static
    address 192.168.60.4
    netmask 255.255.255.0

# ens38 : reseau HA, cote serveurs web
auto ens38
iface ens38 inet static
    address 10.0.0.1
    netmask 255.255.255.0
```

*/etc/hosts sur SRVHAPROXY (lignes IPv6 par défaut conservées en dessous)*

```
127.0.0.1       localhost
127.0.1.1       SRVHAPROXY
192.168.60.4    SRVHAPROXY
10.0.0.2        SRVWEB1
10.0.0.3        SRVWEB2
```

*Fin de /etc/haproxy/haproxy.cfg : sections ajoutées (port 80 au lieu de « 80 ou 443 »)*

```
# ... (sections global et defaults d'origine inchangées) ...

frontend site_web
    bind *:80
    option httpclose
    default_backend serveurs

backend serveurs
    balance roundrobin
    server srvweb1 10.0.0.2:80 weight 1 check
    server srvweb2 10.0.0.3:80 weight 1 check

listen stats
    bind *:8404
    mode http
    stats enable
    stats uri /statshaproxy
    stats auth admin:@admin2627@
    stats refresh 20s
```

### e) Réponses aux questions de l'énoncé

**Rôle du fichier `/etc/hosts` de SRVHAPROXY :** même rôle que sur les serveurs web (annuaire local prioritaire sur le DNS). `192.168.60.4 SRVHAPROXY` désigne son adresse côté clients ; `10.0.0.2 SRVWEB1` et `10.0.0.3 SRVWEB2` permettent de désigner les serveurs web par leur nom.

**État du service après installation (`systemctl status haproxy`).** À l'installation, Debian démarre HAProxy avec sa configuration par défaut, qui ne contient aucun frontend ni backend : le service est **active (running)** mais ne répartit encore rien. Après l'ajout des sections, la figure 18 montre le service **active (running)** et **enabled** (démarrage automatique).

**Rôle de chaque section et paramètre de `haproxy.cfg` :**

| Élément | Rôle |
|---|---|
| `frontend site_web` | Section qui reçoit les requêtes des clients (`site_web` est son nom). |
| `bind *:80` | Écoute sur toutes les adresses de SRVHAPROXY, port 80. |
| `option httpclose` | Ferme la connexion après chaque requête : chaque rechargement est une nouvelle requête, ce qui rend l'alternance visible. |
| `default_backend serveurs` | Envoie les requêtes au backend nommé `serveurs`. |
| `backend serveurs` | Section qui décrit le groupe de serveurs web. |
| `balance roundrobin` | Algorithme de répartition : chacun son tour. |
| `server srvweb1 10.0.0.2:80` | Un serveur du groupe : nom interne, adresse IP et port. |
| `weight 1` | Poids du serveur : avec 1 et 1, répartition égale ; avec 2 et 1, SRVWEB1 reçoit deux fois plus de requêtes. |
| `check` | Vérification régulière du serveur (health check) ; s'il tombe, il ne reçoit plus de requêtes. |
| `listen stats` | Section combinant frontend et backend, ici pour la page de statistiques. |
| `bind *:8404` | La page de statistiques écoute sur le port 8404. |
| `mode http` | HAProxy analyse le trafic au niveau HTTP (couche 7 du modèle OSI). |

| Élément | Rôle |
|---|---|
| `stats enable` | Active la page de statistiques. |
| `stats uri /statshaproxy` | Chemin de la page dans l'URL. |
| `stats auth admin:@admin2627@` | Identifiant `admin` et mot de passe `@admin2627@` demandés à l'ouverture de la page. |
| `stats refresh 20s` | La page se recharge automatiquement toutes les 20 secondes. |

**Commande de vérification.** `haproxy -c -f /etc/haproxy/haproxy.cfg`. L'option `-c` (*check*) vérifie la configuration sans démarrer le service ; `-f` (*file*) indique le fichier à vérifier. Résultat attendu : « Configuration file is valid ». **Ce qui a été constaté :** la première tentative a échoué (figure 17), voir la partie « En cas de problème ».

**Commande pour relancer le service :** `systemctl restart haproxy`.

### f) Résultat attendu

- `hostnamectl` : `Static hostname: SRVHAPROXY` ; `ip -br a` : `ens37` en `192.168.60.4/24` et `ens38` en `10.0.0.1/24`, toutes **UP** ;
- pings vers SRVWEB1 et SRVWEB2 : 4 réponses ;
- `haproxy -c -f …` : « Configuration file is valid » ;
- `systemctl status haproxy` : **active (running)**.

### g) Captures

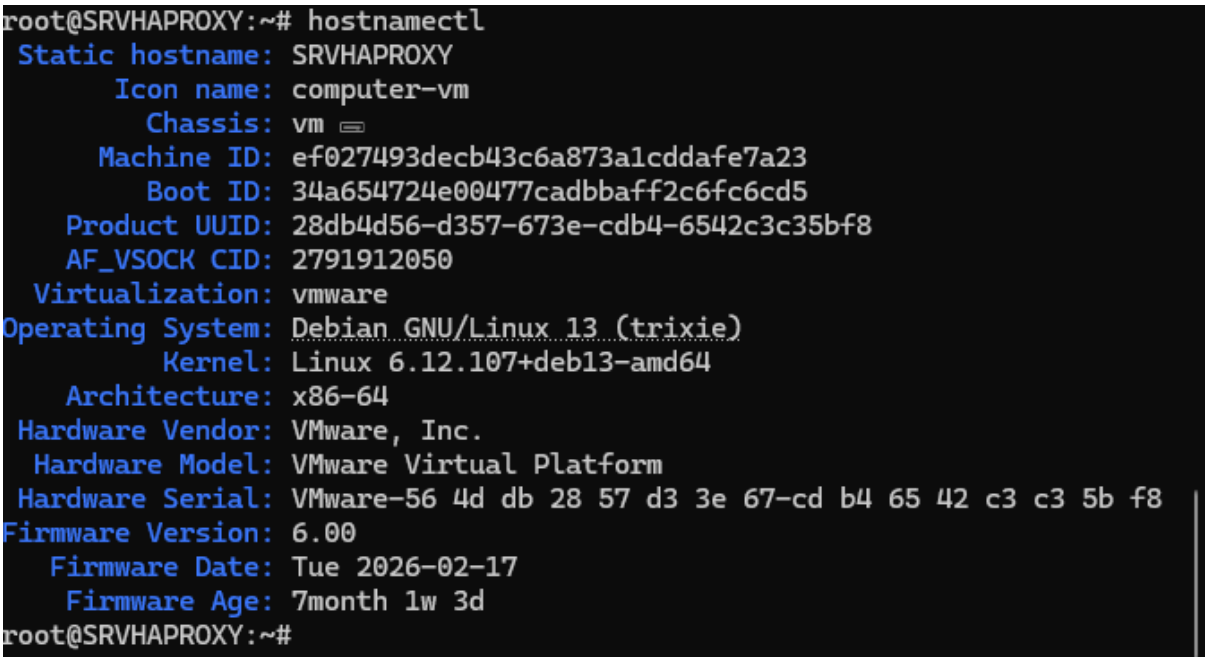



*Figure 10 – Nom d'hôte de SRVHAPROXY*

**Ce que montre la capture.** `hostnamectl` affiche `Static hostname: SRVHAPROXY`, `Debian GNU/Linux 13 (trixie)`, virtualisation `vmware`. Le Machine ID (`ef027493…`) est **identique** à celui des serveurs web du TP3.

**Rôle de l'étape.** Donner à la VM clonée le nom du répartiteur.

**Ce que cela valide.** SRVHAPROXY est renommé et tourne sous l'environnement prévu. Le Machine ID identique est un effet du clonage (perspective en conclusion).

Les trois figures suivantes documentent un **incident réseau** survenu après le clonage. Un premier `ping -c 4 SRVWEB1` depuis SRVHAPROXY avait échoué. Relevé texte (pas de capture) :

*Diagnostic : « From 10.0.0.3 » = la machine qui émet l'erreur est SRVHAPROXY elle-même*

```
root@SRVHAPROXY:~# ping -c 4 SRVWEB1
PING SRVWEB1 (10.0.0.2) 56(84) bytes of data.
From SRVWEB2 (10.0.0.3) icmp_seq=1 Destination Host Unreachable
4 packets transmitted, 0 received, +3 errors, 100% packet loss

root@SRVHAPROXY:~# ip a          (extrait)
3: ens37: ... inet 10.0.0.3/24 ...      <-- anormal : devait être 192.168.60.4
4: ens38: ... inet 10.0.0.1/24 ...
```

Deux cartes se trouvaient dans le sous-réseau `10.0.0.0/24` (`ens37` et `ens38`) et `ens37` reprenait l'adresse de SRVWEB2 : Linux envoyait le trafic par la mauvaise carte, branchée sur le réseau des clients, où personne ne répond. La correction du fichier `interfaces` a alors fait apparaître deux nouvelles erreurs.

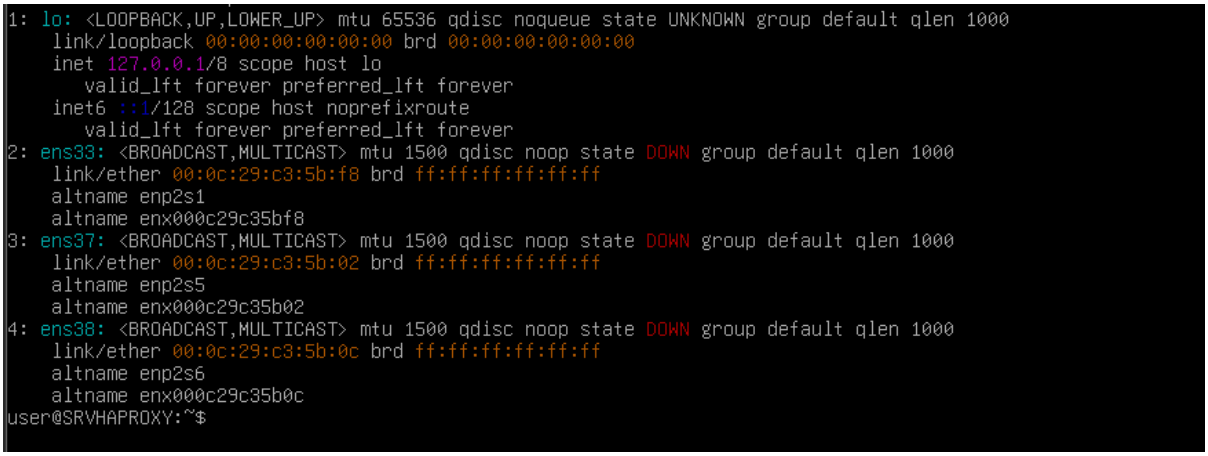



***Figure 11 – Tentative ratée : toutes les cartes DOWN***

**Ce que montre la capture.** `ip a` sur `user@SRVHAPROXY` : `ens33`, `ens37` et `ens38` sont **DOWN**, sans adresse IP. Les indicateurs sont `<BROADCAST,MULTICAST>` : le mot `UP` est absent, donc Linux n'a même pas tenté d'activer les cartes.

**Rôle de l'étape.** Diagnostiquer pourquoi le réseau ne se lève plus après la modification du fichier `interfaces`.

**Ce que cela valide.** Ce n'est pas un problème de câble (qui donnerait `NO-CARRIER`) mais de configuration. La cause est visible à la figure 12.

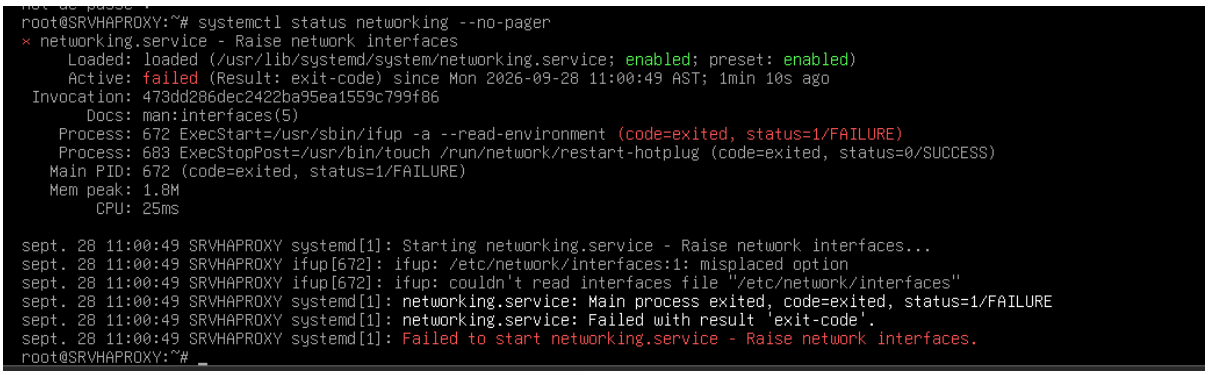



*Figure 12 – Tentative ratée : networking.service en échec*

**Ce que montre la capture.** `systemctl status networking` : `Active: failed`, code `status=1/FAILURE`, et le journal indique `ifup: /etc/network/interfaces:1: misplaced option` puis `couldn't read interfaces file`.

**Rôle de l'étape.** Lire le message d'erreur exact du service réseau.

**Ce que cela valide.** L'erreur est à la **ligne 1** du fichier : une option se trouve hors d'un bloc `iface`. Comme ifup abandonne toute la lecture, aucune carte n'est activée.

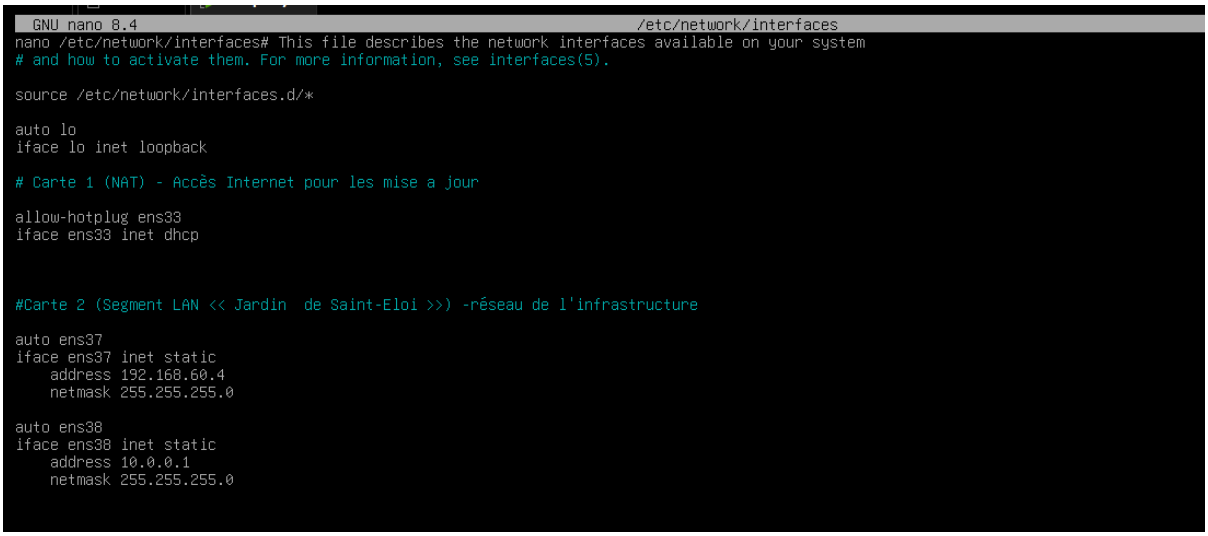



*Figure 13 – Cause : texte parasite en ligne 1 de interfaces*

**Ce que montre la capture.** Dans nano, la ligne 1 est `nano /etc/network/interfaces# This file describes the network interfaces…` : la commande a été collée devant le commentaire d'origine. Le reste du fichier est correct (`ens33` DHCP, `ens37` = 192.168.60.4, `ens38` = 10.0.0.1).

**Rôle de l'étape.** Repérer la ligne fautive dans le fichier.

**Ce que cela valide.** Faute identifiée. La correction consiste à ne garder que `# This file describes…` en ligne 1 ; le résultat est visible à la figure 14.

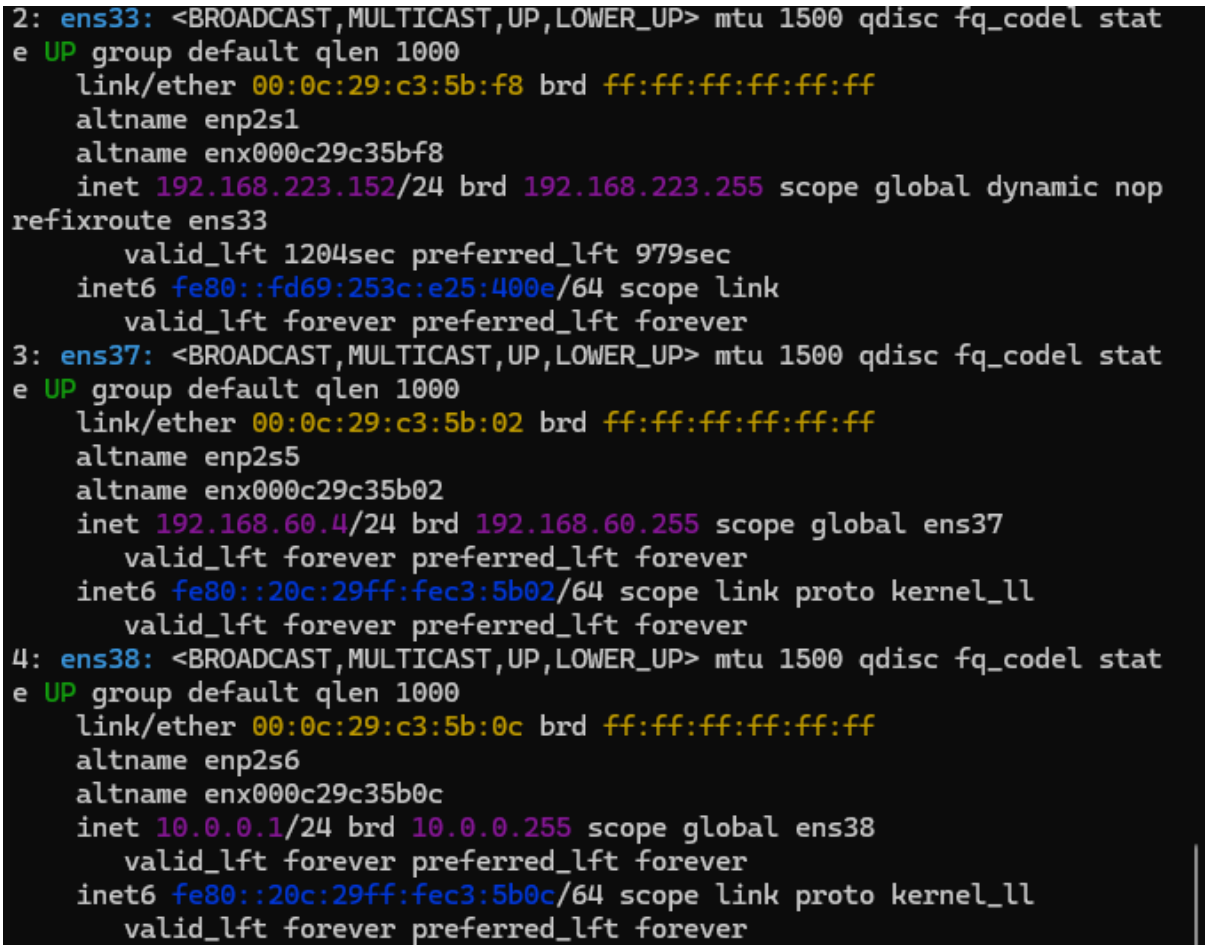



*Figure 14 – Adresses IP de SRVHAPROXY après correction*

**Ce que montre la capture.** `ens33` **UP** en `192.168.223.152/24` (DHCP) ; `ens37` **UP** en `192.168.60.4/24` (MAC `00:0c:29:c3:5b:02`) ; `ens38` **UP** en `10.0.0.1/24` (MAC `00:0c:29:c3:5b:0c`).

**Rôle de l'étape.** Contrôler le plan d'adressage de SRVHAPROXY, relié à la fois aux clients et au réseau HA.

**Ce que cela valide.** Le répartiteur est correctement adressé : `192.168.60.4` côté clients, `10.0.0.1` côté HA. Le conflit d'adresse et l'erreur du fichier `interfaces` sont résolus.

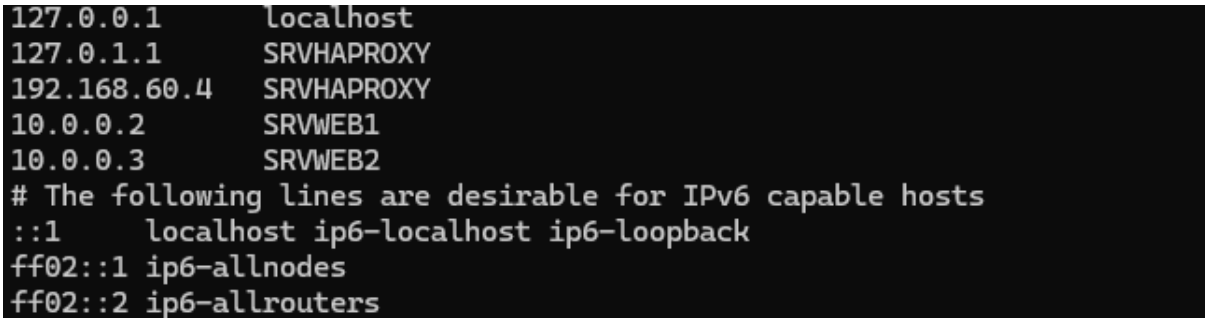



*Figure 15 – Fichier /etc/hosts de SRVHAPROXY*

**Ce que montre la capture.** `localhost` en `127.0.0.1`, `SRVHAPROXY` en `127.0.1.1` et `192.168.60.4`, `SRVWEB1` en `10.0.0.2`, `SRVWEB2` en `10.0.0.3`, suivis des lignes IPv6 par défaut.

**Rôle de l'étape.** Permettre à SRVHAPROXY de désigner les serveurs web par leur nom.

**Ce que cela valide.** La résolution de noms locale est conforme à l'énoncé, avec l'adresse `192.168.60.4` du réseau réel.

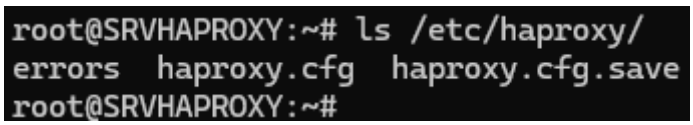



*Figure 16 – Sauvegarde haproxy.cfg.save*

**Ce que montre la capture.** `ls /etc/haproxy/` liste `errors`, `haproxy.cfg` et `haproxy.cfg.save`.

**Rôle de l'étape.** Conserver la configuration d'origine avant modification.

**Ce que cela valide.** La sauvegarde existe : un retour arrière est possible en copiant `haproxy.cfg.save` sur `haproxy.cfg`.

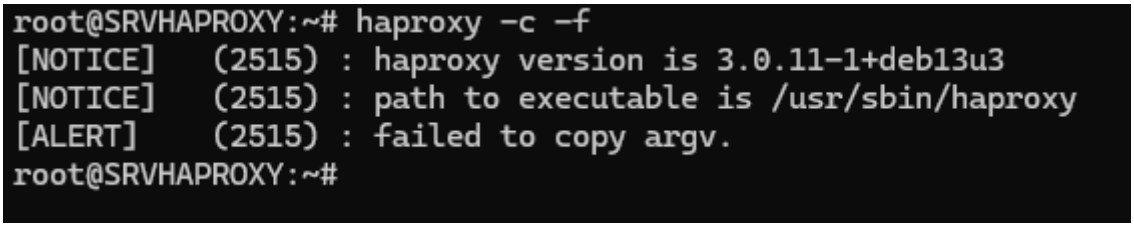



*Figure 17 – Tentative ratée : haproxy -c -f*

**Ce que montre la capture.** `haproxy -c –f` affiche la version `3.0.11-1+deb13u3`, le chemin `/usr/sbin/haproxy` puis `[ALERT] (2515) : failed to copy argv.` La commande n'indique aucun chemin de fichier après `-f`, et le tiret de `–f` est un tiret long recopié de l'énoncé.

**Rôle de l'étape.** Vérifier la validité de la configuration avant de relancer le service.

**Ce que cela valide.** Ce n'est pas une erreur de configuration mais une erreur de saisie : HAProxy n'a pas pu lire ses arguments. La commande correcte est `haproxy -c -f /etc/haproxy/haproxy.cfg`. La figure 20 prouve que la configuration a été chargée.

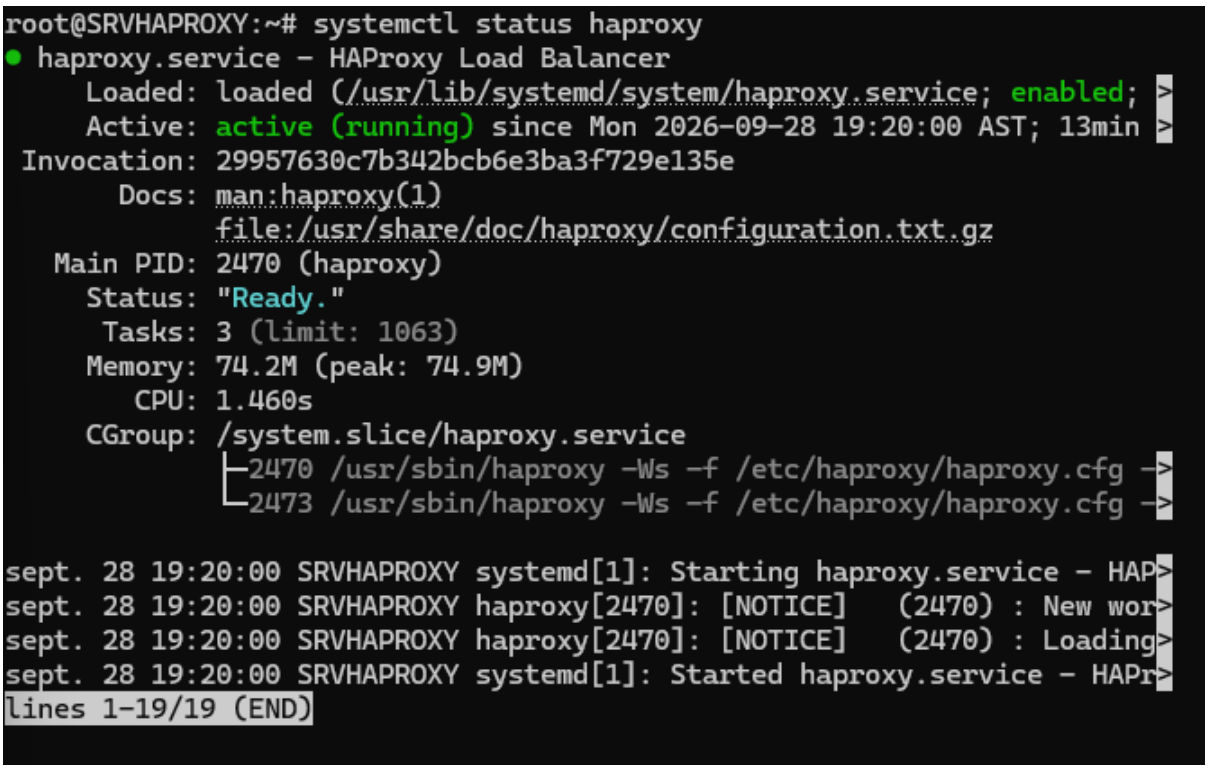



*Figure 18 – État du service HAProxy*

**Ce que montre la capture.** `systemctl status haproxy` : `Loaded … enabled`, **`Active: active (running)`** depuis le 28/09/2026 à 19:20:00, statut « Ready », 3 tâches, processus `/usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg`.

**Rôle de l'étape.** Vérifier que le service tourne et démarre automatiquement.

**Ce que cela valide.** HAProxy est actif et activé au démarrage avec `/etc/haproxy/haproxy.cfg`. Les sections `site_web`, `serveurs` et `stats` de la figure 20 prouvent que ce processus utilise la configuration modifiée.

### h) Point de contrôle

*SRVHAPROXY*

```
haproxy -c -f /etc/haproxy/haproxy.cfg
systemctl is-active haproxy
ss -tlnp | grep haproxy
```

Attendu : « Configuration file is valid », `active`, et HAProxy en écoute sur les ports `80` et `8404`.

### i) En cas de problème

| Symptôme | Cause probable | Diagnostic / correction |
|---|---|---|
| Ping vers un serveur web : `Destination Host Unreachable` avec « From » = adresse de SRVHAPROXY | Deux cartes dans le même sous-réseau ; mauvaise adresse sur `ens37` | `ip -br a`, `ip route` (une seule route `10.0.0.0/24`, via `ens38`) ; corriger `interfaces` (figures 11 à 14). |
| Toutes les cartes DOWN, sans UP | `networking.service` en échec | `systemctl status networking --no-pager` ; `journalctl -u networking -b`. |
| `misplaced option` ligne N | Texte parasite ou option hors bloc `iface` | `cat -A /etc/network/interfaces` ; réécrire la ligne. |
| `haproxy -c` : `failed to copy argv` | Tiret long, ou chemin absent après `-f` | Retaper `haproxy -c -f /etc/haproxy/haproxy.cfg`. |

| Symptôme | Cause probable | Diagnostic / correction |
|---|---|---|
| `cannot bind socket [0.0.0.0:80]` | Apache tourne encore sur SRVHAPROXY | `systemctl stop apache2` et `systemctl disable apache2`. |
| `apt` ne trouve pas haproxy / pas d'Internet | `ens33` non activée | `ip -br a` ; `systemctl restart networking`. |

## 4.4 Étape 4 : tests de la répartition de charge

### a) Objectif

Prouver, depuis Poste0, que HAProxy répartit les requêtes entre les deux serveurs (scénario 1) et que le site reste disponible si l'un des serveurs tombe (scénario 2). C'est la phase de recette.

### b) Prérequis

- Étape 3 validée : HAProxy actif.
- Apache actif sur SRVWEB1 et SRVWEB2.
- Poste0 sur le réseau `192.168.60.0/24` ; son fichier `C:\Windows\System32\drivers\etc\hosts` contient `192.168.60.4  www.jardinsteloi.fr` (déjà présent depuis le TP3, puisque la .4 est désormais SRVHAPROXY).

### c) Procédure

**A. Page de statistiques.**

1. **Sur Poste0**, ouvrez le navigateur sur `http://192.168.60.4:8404/statshaproxy`.
2. Saisissez l'identifiant `admin` et le mot de passe `@admin2627@` (définis par `stats auth` dans `haproxy.cfg`).
3. Vérifiez que `srvweb1` et `srvweb2` sont **verts** (UP).

**B. Scénario 1 : alternance des requêtes.**

1. **Sur Poste0**, ouvrez `http://www.jardinsteloi.fr` en navigation privée (**Ctrl+Maj+P**) et actualisez avec **Ctrl+F5** plusieurs fois : la page alterne entre « site n°1 » et « site n°2 ».
2. Pour une preuve sur un seul écran, envoyez 6 requêtes à la suite depuis l'invite de commandes Windows :

*Poste0 (cmd)*

```
for /L %i in (1,1,6) do curl -s http://www.jardinsteloi.fr
```

3. Actualisez la page de statistiques et comparez la colonne **Sessions > Total** de `srvweb1` et `srvweb2`.

**C. Scénario 2 : panne d'un serveur.**

1. **Sur SRVWEB1**, simulez la panne :

*SRVWEB1*

```
systemctl stop apache2
```

2. **Sur Poste0**, actualisez la page de statistiques : `srvweb1` passe au **rouge** (DOWN) grâce au paramètre `check`.
3. **Sur Poste0**, rechargez `http://www.jardinsteloi.fr` (Ctrl+F5, plusieurs fois) : seule la page « site n°2 » s'affiche.
4. **Sur SRVWEB1**, remettez le service en marche :

*SRVWEB1*

```
systemctl start apache2
```

5. Après quelques secondes, `srvweb1` repasse au **vert** et l'alternance reprend.

### d) Contenu des fichiers

Aucun fichier n'est modifié à cette étape. Les identifiants d'accès aux statistiques viennent de la ligne `stats auth admin:@admin2627@` de `/etc/haproxy/haproxy.cfg`.

### e) Réponses aux questions de l'énoncé

**Informations d'authentification :** identifiant **`admin`**, mot de passe **`@admin2627@`**, fournis par `stats auth` dans `/etc/haproxy/haproxy.cfg`. **Deux scénarios proposés :** (1) alternance des requêtes en régime normal, prouvée par des compteurs de sessions égaux ; (2) arrêt d'Apache sur un serveur, avec continuité du service grâce au health check.

### f) Résultat attendu

- Scénario 1 : alternance n°1 / n°2 ; compteurs de sessions égaux pour les deux serveurs.
- Scénario 2 : `srvweb1` en rouge, toutes les réponses viennent de `srvweb2` ; retour au vert après relance d'Apache.

### g) Captures

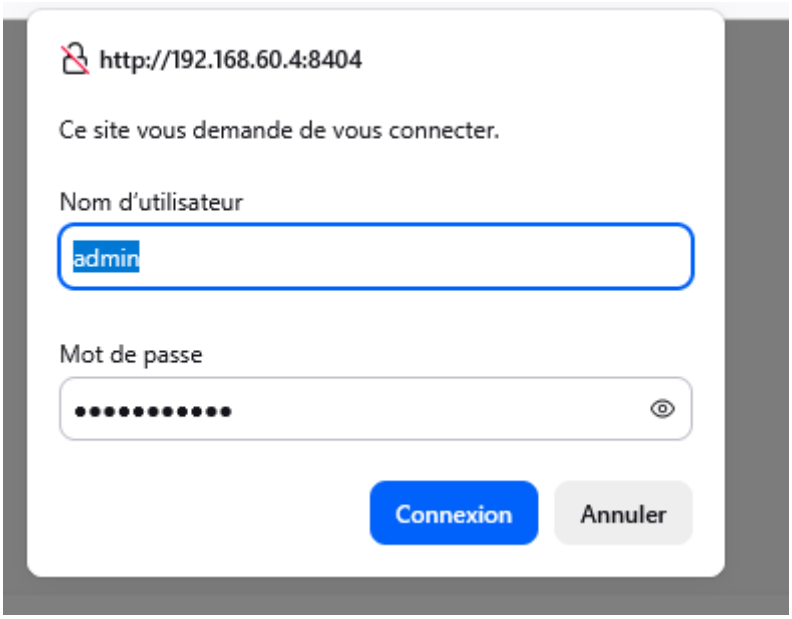



*Figure 19 – Authentification à la page de statistiques*

**Ce que montre la capture.** La boîte de dialogue du navigateur pour `http://192.168.60.4:8404` demande « Nom d'utilisateur » (`admin`) et « Mot de passe » (champ masqué de 11 caractères, soit la longueur de `@admin2627@`).

**Rôle de l'étape.** Contrôler l'accès à la supervision par le `stats auth` du fichier de configuration.

**Ce que cela valide.** La page est protégée : sans identifiants, on n'y accède pas. **Ordre :** dans le fichier de captures, cette image suivait la page de statistiques ; l'authentification précédant nécessairement l'affichage, elle est placée avant.

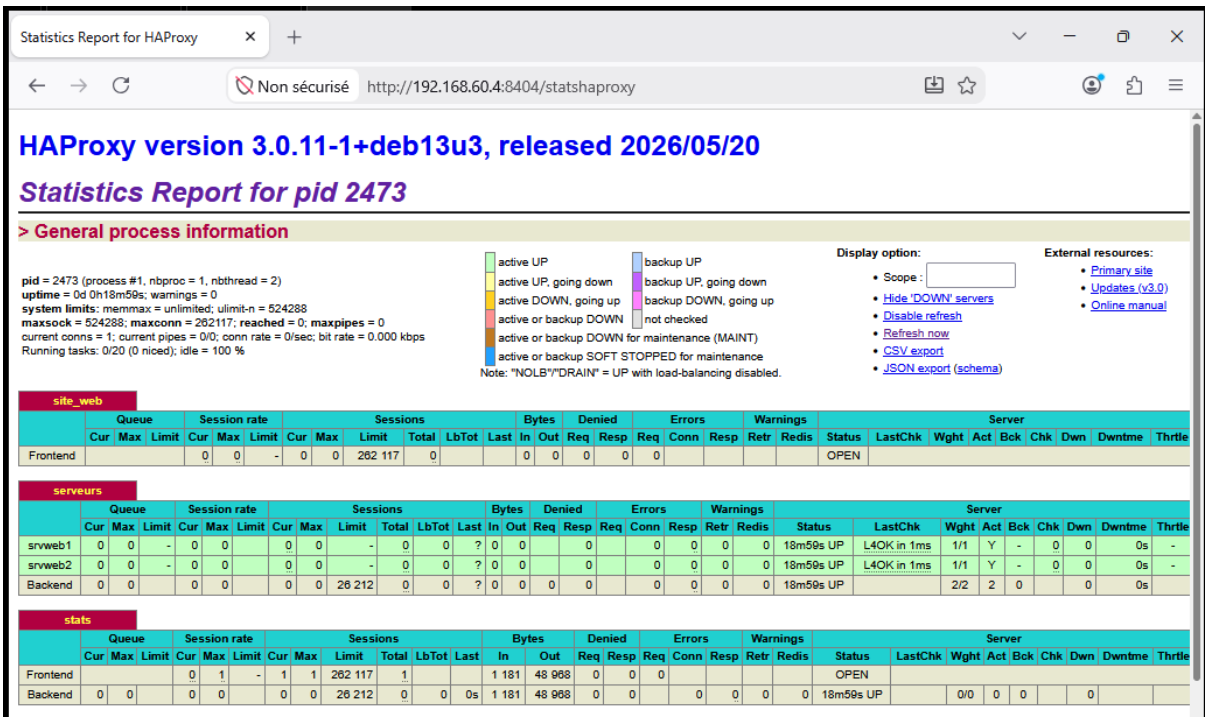



*Figure 20 – Page de statistiques : les deux serveurs UP*

**Ce que montre la capture.** À l'adresse `http://192.168.60.4:8404/statshaproxy` (« HAProxy version 3.0.11-1+deb13u3 », pid 2473, uptime 18 min 59 s). Trois sections : `site_web` (Frontend, `OPEN`), `serveurs` (`srvweb1` et `srvweb2` **18m59s UP**, `L4OK`, poids `1/1`, sessions totales 0) et `stats`.

**Rôle de l'étape.** Superviser l'état des serveurs avant les tests.

**Ce que cela valide.** HAProxy fonctionne avec la configuration demandée, et ses health checks voient les deux serveurs vivants. Les compteurs à 0 servent d'état de départ.

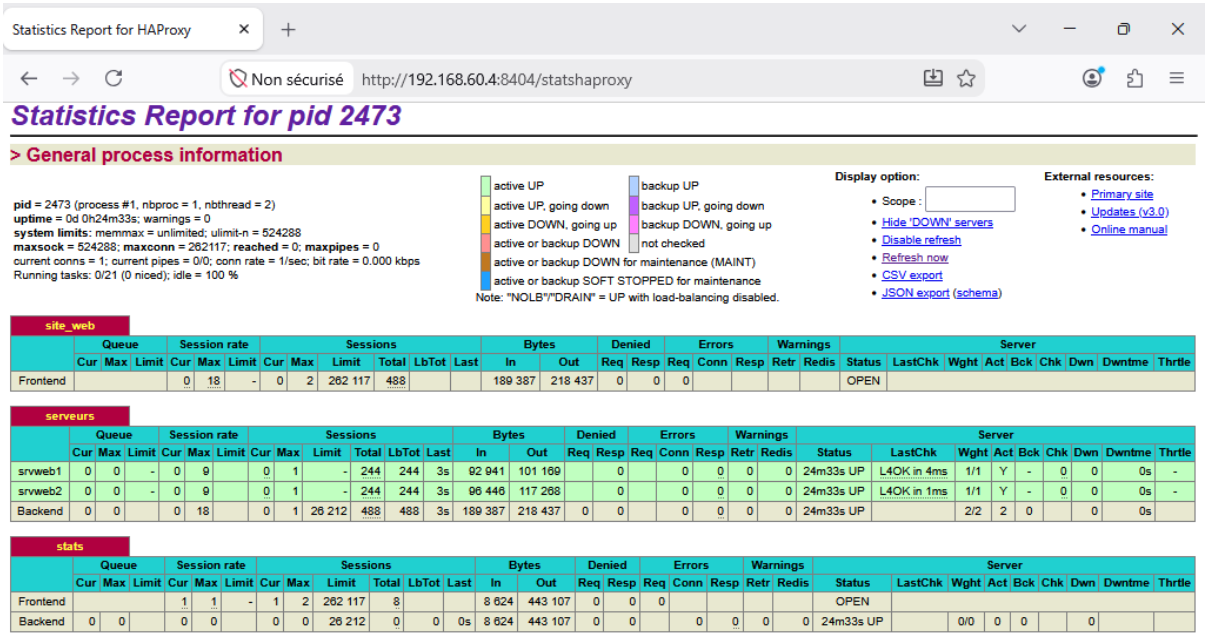



*Figure 21 – Scénario 1 : sessions réparties à égalité*

**Ce que montre la capture.** Après les tests (uptime 24 min 33 s) : `site_web` (Frontend) totalise **488** sessions ; `srvweb1` et `srvweb2` en ont **244 chacun** (`LbTot` 244/244), poids `1/1`, tous deux `24m33s UP`.

**Rôle de l'étape.** Mesurer la répartition réelle des requêtes par serveur.

**Ce que cela valide.** Le round robin fonctionne : 488 requêtes partagées **exactement à égalité** (50 % / 50 %). Les serveurs sont utilisés de façon optimale, ce qui répond au besoin de Mme Bourgeois.

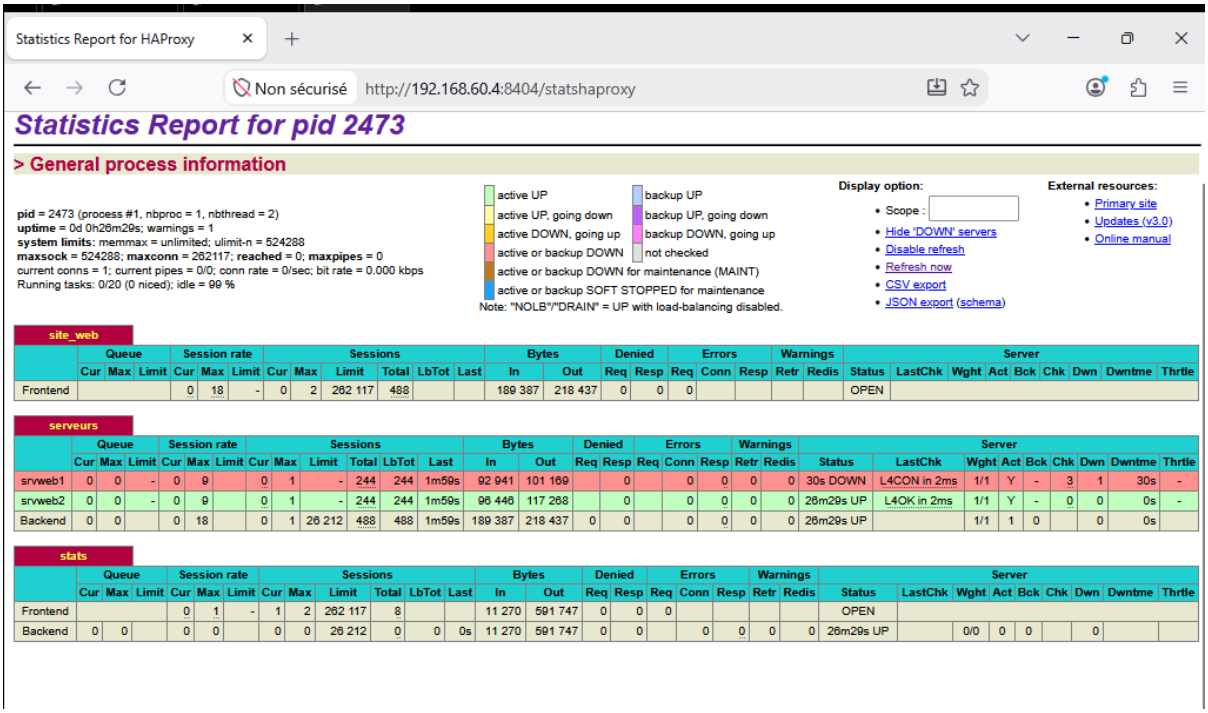



*Figure 22 – Scénario 2 : srvweb1 en panne (DOWN)*

**Ce que montre la capture.** La ligne `srvweb1` est **rouge** : `30s DOWN`, `L4CON in 2ms`, 1 passage en panne. `srvweb2` reste vert (`26m29s UP`). Les sessions totales n'ont pas bougé (244/244) et le Backend indique 1 serveur actif.

**Rôle de l'étape.** Constater que HAProxy détecte la panne de SRVWEB1.

**Ce que cela valide.** Le paramètre `check` fonctionne : HAProxy a retiré `srvweb1` du groupe et n'y envoie plus de requêtes. `L4CON` signifie que la connexion TCP vers Apache est refusée.




*Figure 23 – Site servi par SRVWEB2 pendant la panne*

**Ce que montre la capture.** Navigation privée sur `http://www.jardinsteloi.fr` : « Bienvenue sur le site n°2 des Jardins de Saint-Eloi ».

**Rôle de l'étape.** Vérifier, côté client, que le site reste accessible.

**Ce que cela valide.** Le site répond alors que SRVWEB1 est arrêté (figure 22). Cette image seule ne prouve pas l'absence d'alternance (n°2 apparaît aussi en régime normal).

### h) Point de contrôle

Après avoir relancé Apache sur SRVWEB1 : la page de statistiques montre les deux serveurs **verts** et la boucle `curl` alterne de nouveau entre n°1 et n°2.

### i) En cas de problème

| Symptôme | Cause probable | Diagnostic / correction |
|---|---|---|
| Page de statistiques inaccessible | Mauvaise URL ou port | Utiliser `http://192.168.60.4:8404/statshaproxy` ; `ss -tlnp \| grep 8404` sur SRVHAPROXY. |
| Identifiants refusés | Mot de passe différent de `stats auth` | `grep "stats auth" /etc/haproxy/haproxy.cfg`. |
| srvweb1 / srvweb2 en rouge en permanence | Apache arrêté ou réseau HA coupé | `systemctl status apache2` ; `ping -c 4 SRVWEB1` depuis SRVHAPROXY. |
| La page n'alterne pas dans le navigateur | Cache ou connexion réutilisée | Navigation privée, Ctrl+F5, ou boucle `curl`. |
| Le nom ne répond pas mais `192.168.60.4` oui | Fichier `hosts` de Poste0 absent | Ajouter `192.168.60.4 www.jardinsteloi.fr` puis `ipconfig /flushdns`. |

# 5. Difficultés rencontrées et solutions

| Difficulté | Diagnostic | Solution | Figure |
|---|---|---|---|
| Ping SRVHAPROXY → SRVWEB1 : `Destination Host Unreachable`, « From SRVWEB2 (10.0.0.3) » | L'expéditeur de l'erreur est SRVHAPROXY : `ens37` portait `10.0.0.3` ; deux cartes dans `10.0.0.0/24` | `ens37` = `192.168.60.4` dans `interfaces` | Relevé texte, figure 14 |
| Toutes les cartes DOWN après modification du fichier | `networking.service` en échec, `misplaced option` ligne 1 | Lire `systemctl status networking` | figure 11, figure 12 |
| Commande collée en ligne 1 de `interfaces` | Texte parasite avant le commentaire d'origine | Supprimer `nano /etc/network/interfaces` en ligne 1 ; `systemctl restart networking` | figure 13 → figure 14 |
| `haproxy -c –f` : `failed to copy argv` | Tiret long copié depuis le PDF et chemin de fichier absent | Retaper `haproxy -c -f /etc/haproxy/haproxy.cfg` | figure 17 |
| Adresse de SRVWEB2 affichée à `10.0.0.2` | Conflit possible avec SRVWEB1 ; capture antérieure à la correction ou erreur de saisie | Vérifier `ip -br a` ; refaire les captures | figure 3, figure 9 |
| Poste0 ne joint pas les serveurs web | Réseau HA isolé (voulu) | Tests directs depuis SRVHAPROXY avec `curl` | figure 8, figure 9 |
| Même Machine ID que les autres VM | Effet du clonage VMware | `rm /etc/machine-id && systemd-machine-id-setup` puis redémarrage | figure 10 |

## 6. Conclusion et perspectives

HAProxy est en place devant les deux serveurs web. Les statistiques montrent une répartition parfaitement égale (244 requêtes pour chaque serveur sur 488), et l'arrêt d'Apache sur SRVWEB1 est détecté en quelques secondes : le site reste servi par SRVWEB2. Le besoin de Mme Bourgeois, utiliser chaque serveur de façon optimale tout en gardant la disponibilité, est satisfait. Reste à vérifier l'adresse de SRVWEB2.

**Comparaison avec le TP3 :** Heartbeat (actif/passif) laisse un serveur inutilisé et n'offre que la haute disponibilité ; HAProxy (actif/actif) l'offre aussi mais utilise les deux serveurs et **vérifie le service web lui-même** (`check`), pas seulement la machine.

**Limites et perspectives :**

- **Point de défaillance unique.** SRVHAPROXY est lui-même unique : s'il tombe, tout s'arrête. On le double avec **Keepalived** (adresse IP virtuelle VRRP) et un second répartiteur.
- **Sécurité.** Passer en **HTTPS** (port 443, certificat sur HAProxy), utiliser un mot de passe fort pour les statistiques et restreindre l'accès au port 8404.
- **Contenu identique.** Synchroniser automatiquement les sites des deux serveurs (`rsync`, stockage partagé), comme le montrent les pages différentes des figures 7 et 6.
- **Adresse réelle du client.** Ajouter `option forwardfor` pour que les journaux des serveurs web voient l'adresse du client plutôt que celle de SRVHAPROXY.
- **Clonage propre.** Régénérer le Machine ID et les clés SSH d'hôte après chaque clonage.

# 7. Aide-mémoire des commandes

| Commande | Machine | Rôle |
|---|---|---|
| `su -` | Toutes (Linux) | Passer en root avec son environnement complet. |
| `hostnamectl set-hostname NOM` / `hostnamectl` | Serveurs | Changer / afficher le nom d'hôte. |
| `nano <fichier>` | Serveurs | Éditer un fichier (Ctrl+O enregistrer, Ctrl+X quitter). |
| `systemctl restart networking` | Serveurs | Appliquer `/etc/network/interfaces`. |
| `systemctl status networking --no-pager` | SRVHAPROXY | Voir l'état du service réseau et ses erreurs. |
| `ip a` / `ip -br a show ens37` | Serveurs | Afficher les cartes et leurs adresses. |
| `ip route` | Serveurs | Afficher les routes et la passerelle. |
| `ping -c 4 <cible>` | Serveurs | Tester la connectivité (4 paquets). |
| `systemctl stop\|start\|restart\|enable\|disable <service>` | Serveurs | Arrêter, démarrer, redémarrer, activer / désactiver au démarrage. |
| `systemctl status\|is-active <service>` | Serveurs | Afficher l'état d'un service. |
| `apache2ctl configtest` | SRVWEB1, SRVWEB2 | Vérifier la syntaxe d'Apache. |
| `apt update` / `apt upgrade -y` | SRVHAPROXY | Mettre à jour la liste des paquets / le système. |
| `apt install haproxy -y` / `apt install curl -y` | SRVHAPROXY | Installer HAProxy / curl. |
| `curl http://<adresse>` | SRVHAPROXY, Poste0 | Afficher le code HTML renvoyé par un serveur web. |
| `cp haproxy.cfg haproxy.cfg.save` | SRVHAPROXY | Sauvegarder la configuration. |
| `ls /etc/haproxy/` | SRVHAPROXY | Lister les fichiers du dossier. |
| `haproxy -c -f /etc/haproxy/haproxy.cfg` | SRVHAPROXY | Vérifier la configuration (`-c` check, `-f` file) sans démarrer. |
| `ss -tlnp` | SRVHAPROXY | Lister les ports TCP en écoute et leur programme. |
| `cat -A <fichier>` | Serveurs | Afficher un fichier avec ses caractères invisibles. |
| `journalctl -u <service> -n 30 --no-pager` | Serveurs | Afficher le journal d'un service. |
| `for /L %i in (1,1,6) do curl -s <URL>` | Poste0 (cmd) | Envoyer 6 requêtes à la suite. |
| `ipconfig /flushdns` | Poste0 | Vider le cache DNS de Windows. |

# 8. Sitographie

Aucun site n'a été fourni comme source personnelle. Les références ci-dessous sont les documentations officielles des logiciels utilisés (à confirmer par l'étudiant avant remise).

| Source | Adresse | Étape concernée |
|---|---|---|
| Site officiel de HAProxy | https://www.haproxy.org/ | Étape 3 |
| Manuel de configuration HAProxy 3.0 (frontend, backend, balance, check, stats) | https://docs.haproxy.org/3.0/configuration.html | Étapes 3 et 4 |
| Paquet Debian haproxy (trixie) | https://packages.debian.org/trixie/haproxy | Étape 3 |
| Apache HTTP Server 2.4 : hôtes virtuels | https://httpd.apache.org/docs/2.4/vhosts/ | Étape 2 |
| Wiki Debian : configuration réseau | https://wiki.debian.org/NetworkConfiguration | Étapes 1 et 3 |
| curl : documentation | https://curl.se/docs/ | Étape 2 |

**Autres ressources :** énoncés des TP n°3 et n°4 (bloc 3) ; assistance au dépannage par l'assistant IA Claude (Anthropic), le 28/09/2026. Ajoutez les pages web que vous avez réellement consultées.